Aufgabenstellung zur Masterarbeit

Danje Petersen

October 30, 2025

1 Motivation

Modern computers come with a set of processors that can run simultaneously. To utilize this for their advantage, developers started to design their programs to use multiple concurrent running threads. Since these threads might still operate on the same memory it is possible to encounter problems like data-races when executing these programs. Data races occur when two or more threads access the same memory space and at least one of them uses a write operation. The classic example is the following:
TODO ▶*Grafik*◀
Since this might lead to inconsistent results and therefore unpredictable behaviour, programming languages and cpu architectures provide memory consistency models. If the developer confines themselves to these rules, they get certain guarantees in return. These memory consistency models describe how threads are allowed to interact with each other when it comes to shared memory.

- Concurrency als Problem evtl mit Beispiel

2 Current Developements

In 2014 Alglave et al. proposed an axiomatic framework for modelling weak memory and the corresponding language CAT [1]. Axiomatic models work by constraining relations on memory access. Some very basic examples for these relations are the *program order* (po) which represents the sequence of events of a single thread and *read-from* (rf), displaying for every read which write determined the value it sees. Upon these and some more base relations, memory consistency models can be expressed with CAT as can be seen in fig. 1.

While the tool that Alglave et al. developed allows to test certain programs in a given memory model, verifying these models is a different issue. Verification of the memory consistency model means checking if a given model guarantees certain properties for every program that can be executed under it. The first tool to tackle this issue was KATER [**Kok+23**] which covered only a fragment of CAT that could be transformed into Kleene algebra with tests.

- TODO ▶*KATER Tool*◀
- TODO ▶*KATI Tool*◀
- TODO ▶*RAT-CAT-SAT*◀

$$
\begin{aligned}
\langle cat \rangle &:= \texttt{let}\ [\texttt{rec}]\ \langle name \rangle = \langle rel \rangle \mid \langle constraint \rangle \mid \langle cat \rangle\ \langle cat \rangle \\
\langle constraint \rangle &:= \texttt{acyclic}\ \langle rel \rangle \mid \texttt{irreflexive}\ \langle rel \rangle \\
\langle rel \rangle &:= \langle base_rel \rangle \mid \langle rel \rangle.\langle rel \rangle \mid \langle rel \rangle^* \mid \langle rel \rangle \cup \langle rel \rangle \mid \langle rel \rangle \cap \langle rel \rangle \\
&\quad \mid \langle rel \rangle^{-1} \mid \langle rel \rangle^+ \mid \langle rel \rangle? \mid \langle rel \rangle \setminus \langle rel \rangle \mid \sim \langle rel \rangle \mid \langle base_set \rangle \times \langle base_set \rangle \\
\langle base_rel \rangle &:= \text{po} \mid \text{rf} \mid \text{rmw} \mid \text{loc} \mid \text{ext} \mid \text{id} \mid \emptyset \\
\langle base_set \rangle &:= \text{R} \mid \text{W} \mid \text{F} \mid \text{B} \mid \text{M} \mid \text{IW} \mid \text{E}
\end{aligned}
$$

Figure 1: The CAT language. $\langle rel \rangle^+$ is the transitive closure, $\langle rel \rangle?$ the reflexive closure, $\langle rel \rangle^*$ the reflexive and transitive closure, $\sim \langle rel \rangle$ the complement [1]

3 Aufgabe

- Beweissystem erstellen, dass
- Invarianten benutzt
- Inklusion direkt behandelt
- nicht auf Eventebene auflöst

References

[1] Jade Alglave, Luc Maranget, and Michael Tautschnig. "Herding Cats: Modelling, simulation, testing and data mining for weak memory". In: *ACM Transactions on Programming Languages and Systems* 36.2 (2014), pp. 1–74. ISSN: 0164-0925. DOI: 10.1145/2627752.